Student Name: Sujith M

Register Number: 715523244050

Institution: PSG Institute of Technology and applied research

Department: Computer Science and Business Systems

Date of Submission: 26.04.25

1.Problem Statement

Credit card fraud inflicts **$32B+ in annual losses globally**, as traditional rule-based systems fail to detect **25-40% of sophisticated scams** (AFP), including AI-driven attacks and synthetic identity fraud. These systems also generate excessive false positives—blocking **1 in 5 legitimate transactions**—eroding customer trust while letting **< 0.1% of fraudulent slips** through due to severe class imbalance. With digital payments demanding **real-time decisions (<100ms)**, outdated methods lack both speed and accuracy, exposing businesses and consumers to escalating risks.

2.Objectives of the Project

- Develop an ML model that achieves >90% recall (to catch most frauds) while keeping false positives <5%
- Use EDA to identify high-risk behaviors (e.g., micro-transactions before large withdrawals, foreign country spikes).
- Output per-transaction risk scores (0-100%) for adaptive decision-making.
- Deploy as an API (**FastAPI/Flask**) or interactive dashboard (**Streamlit**) for demo

3.Scope of the Project

- Analysis of transaction features: Amount, Time, Location, Merchant, and derived behavioral patterns.
- Model focus: Interpretable ML (**XGBoost, Logistic Regression**) with real-time fraud probability outputs.

Limitations:

Data: Restricted to PCA-transformed numerical features (no raw card/user details)**.**

4.Data Sources

- **Source:** Kaggle Credit Card Fraud Dataset (Public)
- **Type:** Static (one-time download)
- **Details:**
  - **284,807** transactions (492 frauds)
  - Features: PCA-transformed (V1-V28), Time, Amount, Class (0/1)

5.High-Level Methodology

Data Collection -

- Primary: Download Kaggle's CSV dataset.
- Augment: Generate synthetic fraud data if needed.
- Test: Simulate real-time transactions via mock API.

Data Cleaning -

- Missing values → Drop or impute (median/mean)
- Duplicates → Remove exact matches
- Class imbalance → Use **SMOTE**/undersampling
- Inconsistent scales → Normalize Amount (**StandardScaler**)

Exploratory Data Analysis (EDA) -

- Fraud/normal trends: Time-series plots, amount distributions.
- Correlations: Heatmap (PCA features vs. fraud).
- Outliers: Clustering + **SHAP** analysis.
- Tools: seaborn, plotly.

Feature Engineering -

- New Features: Transaction frequency/hour, normalized Amount.
- Transformations: Log-scaling skewed features, PCA for dimensionality.
- Metrics: Precision, Recall, F1, AUC-ROC (focus: Recall >90%).
- Validation: Stratified K-fold (5 splits), holdout test set.

Model Building -

- **Algorithms:**
  - Logistic Regression (Baseline interpretability)
  - Random Forest (Handles non-linear patterns)
  - XGBoost (Optimizes imbalanced data)
  - Autoencoder (Anomaly detection for rare frauds)
- **Why Suitable:** Balance accuracy (XGBoost), interpretability(Logistic Regression), and scalability (Autoencoder).

Model Evaluation -

- Recall (Top priority: Catch >90% frauds)
- Precision (Keep false alarms <5%)
- AUC-ROC (Overall performance)

Visualization and Interpretation -

- Fraud vs. Normal: Time-series & amount distribution plots
- Feature Importance: SHAP summary plot (top fraud drivers)
- Model Performance: ROC curve, confusion matrix

Deployment Plan -

- Output: Interactive fraud-detection dashboard
- Tools:

  Frontend: Streamlit (simple UI for predictions)

  Backend: Flask (API endpoint for model scoring)

6.Tools and Technologies

- **Programming Language**: Python
- **Notebook/IDE:** Jupyter Notebook (local) + Google Colab (cloud)

Key Libraries:

- **Data**: pandas, numpy
- **Visualization:** seaborn, plotly
- **ML:** scikit-learn, XGBoost, imbalanced-learn

Deployment :

- **Prototype:** Streamlit (dashboard)
- **API:** Flask (local testing)

7.Team Members and Roles

Name	Role
Madhan Kumaar B S	• Data collection & cleaning • Feature engineering • Model building & optimization
Manoj G	• Prototype development (Streamlit) • API integration (Flask) • Documentation
Sujith M	• EDA & visualization • Insight generation • Model interpretation (SHAP)